MCF OBSERVER · ENGENHARIA APLICADA

DOSSIÊ TÉCNICO / DOCUMENTAÇÃO DE EXECUÇÃO

Arquitetura, Engenharia & Rastreabilidade

Do primeiro inventário de processos à validação após reboot: explicação completa e verificável do painel de observabilidade OpenClaw.

VERSÃO PÚBLICA REVISADA

20 CAPÍTULOS + 4 CAPTURAS

Consolidação
16/10/2026 05:42 (horário de Pernambuco)

Escopo
Linux Mint · systemd · Python · SQLite · Browser

Etapa seguinte
Correção do browser ainda não executada

MCF OBSERVER

Índice de engenharia

20 capítulos estruturados do objetivo à validação, com quatro capturas reais da interface no final.

01 Resumo executivo e compromisso de rastreabilidade

02 Fronteiras do sistema: o que pertence a quem

03 Inventário técnico antes de programar

04 Requisitos e justificativas de tecnologia

05 Construção do backend: responsabilidades concretas

06 Matemática de CPU, memória e disponibilidade

07 Persistência: estrutura da base SQLite

08 Engenharia da interface: experiência e arquitetura visual

09 Implantação pelo systemd e atalho no desktop

10 Segurança, isolamento e limitações operacionais

11 Plano de testes em camadas e critérios de aceitação

12 Experimento controlado de reinicialização

13 Falhas que o relatório preserva sem corrigir

14 Registro de ferramentas utilizadas pela assistência

15 Agentes e LLMs: atribuição factual do trabalho

16 Controle de mudanças, governança e reversibilidade

17 Engenharia da publicação pública

18 Próximo ciclo de engenharia (proposto, ainda não executado)

19 Glossário técnico em linguagem natural

20 Conclusão técnica e fontes de verdade

Leitura orientada: comece pelo resumo executivo (01), veja arquitetura (02 a 10), leia os testes (11 a 13), consulte ferramentas, agentes e modelos (14 a 15) e finalize com as provas visuais.

01 — Resumo executivo e compromisso de rastreabilidade

O **MCF Observer** é um painel web de observabilidade construído para acompanhar o **OpenClaw Gateway**, sua árvore de processos, o subsistema de navegador gerenciado e serviços associados à sessão Linux. Este dossiê documenta *o que foi feito, como, por que, com quais ferramentas, quais evidências foram recolhidas e quais partes ainda não estão corrigidas*.

Marco temporal: 16 de outubro de 2026, em horário de Pernambuco. **Ambiente:** Linux Mint, Cinnamon, X11, notebook pessoal. **Execução:** ferramenta autorizada de acesso remoto, sob solicitação do proprietário. **Estado observado:** painel funcional e persistente; Gateway ativo após uma tentativa inicial malsucedida; navegador gerenciado ainda não recuperado.

ITEM	RESULTADO OBSERVADO	LIMITE DA CONCLUSÃO
Aplicação MCF Observer	Implementada e acessível por HTTP local	Não é um serviço distribuído na nuvem
Serviço do painel	Iniciado automaticamente após reboot	Persistência testada em uma reinicialização
OpenClaw Gateway	Ativo após retry do systemd	Inferência com modelos não testada nesta missão
CPU e RAM	Dados reais atualizados a cada cinco segundos	Amostras, não medições contínuas do kernel
Banco local	SQLite, retenção projetada para 24 horas	Coletor precisa permanecer ativo
Logs systemd	Consultados por unidade, com ocultação conservadora	Não equivale a SIEM completo
Recuperação do browser	Falhou depois de reiniciar, código 2	Possível corrida de boot, ainda hipótese
CDP gerenciado	Porta indisponível	Outra porta Chrome aberta não resolve
Screenshots	Capturas antes e depois do boot	Captura visual não garante capacidade dos agentes

> **Critério de honestidade operacional:** fatos observados, hipóteses e ações não realizadas são registrados em categorias distintas. Nenhuma correção do subsistema navegador foi aplicada neste trabalho.

Analogia para leitura do documento

O **Gateway é a central**, o **coletor é o instrumento**, a **API é o balcão de informações**, o **SQLite é o livro de registros**, o **systemd é o supervisor de serviços**, e a **UI é a sala de controle**. Os termos técnicos são definidos ao longo do texto e reunidos no glossário.

02 — Fronteiras do sistema: o que pertence a quem

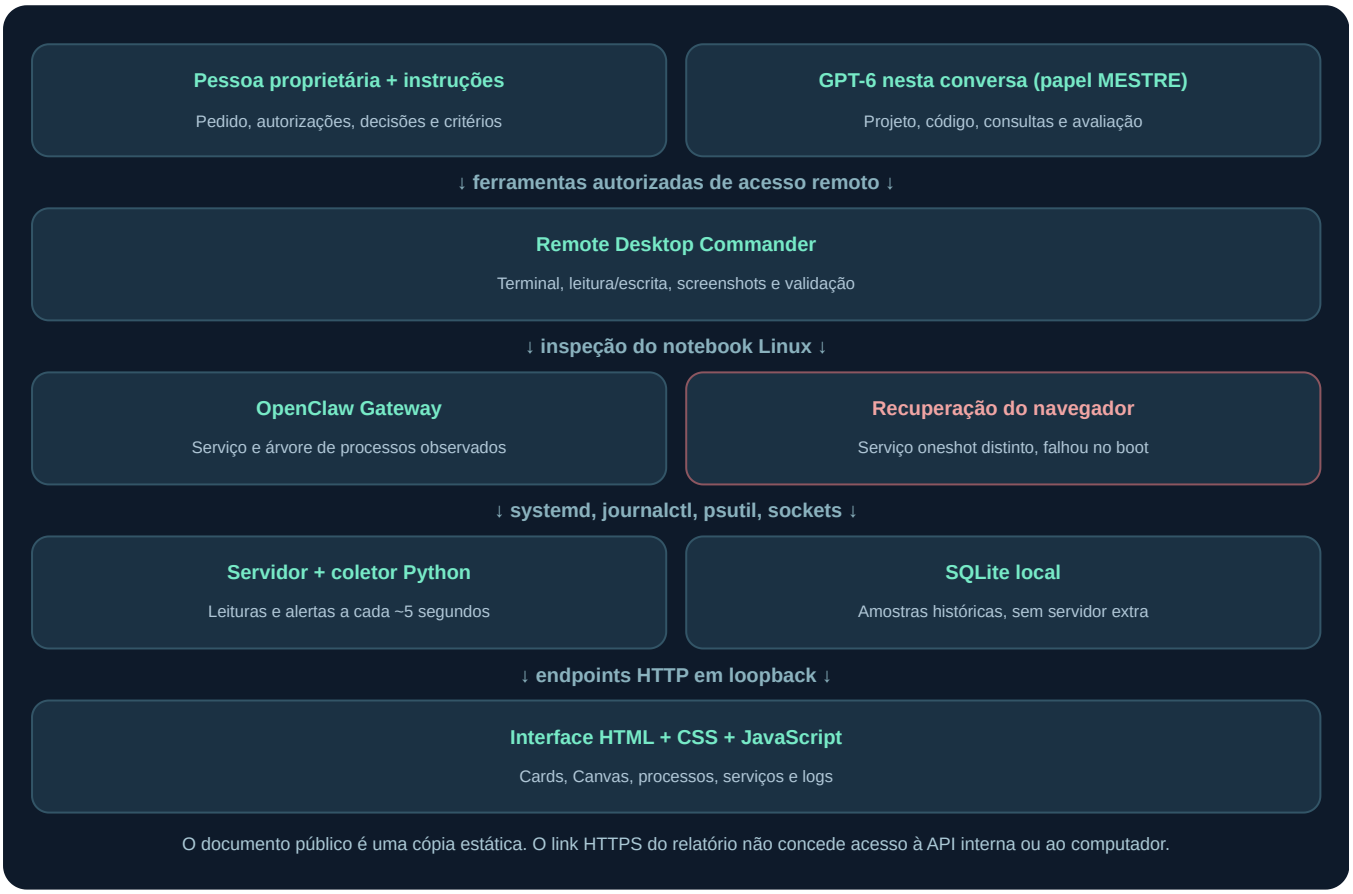
O sistema estudado possui três blocos com responsabilidades separadas.

Bloco A — OpenClaw: produto monitorado. O Gateway mantém processos e plugins, sessões e agentes. O serviço mcf-openclaw-browser-recover.service tenta recuperar o navegador gerenciado. Esta auditoria não modificou a configuração de modelos, agentes ou mecanismo de recuperação.

Bloco B — MCF Observer: aplicação independente criada para ler periodicamente o estado da máquina. Uma falha do painel não deve derrubar o Gateway. O observador não reinicia, corrige ou altera o serviço monitorado.

Bloco C — operador e execução: as decisões e orientações são tomadas nesta conversa. A execução local foi realizada por Desktop Commander Remote, que abriu comandos no notebook. O Google Chrome em modo headless renderizou a UI e gerou capturas. GitHub foi consultado para ler o contrato arquitetural do papel MESTRE; isso não acionou um agente do OpenClaw.

Diagrama: fluxo de dados e controle



Separação de privilégios

O servidor de monitoramento não armazena chave de IA e não requer credenciais do Gateway para avaliar CPU, serviços e portas. A UI acessa somente a API do próprio painel. O serviço HTTP é vinculado a loopback, não à rede local; publicar o PDF em hospedagem estática não publica os endpoints da aplicação.

03 — Inventário técnico antes de programar

A implementação começou pela identificação do comportamento **real** da máquina. Um serviço habilitado não é necessariamente um serviço funcionando; um ícone aberto não é prova de uma API pronta.

1. **Conectar ao dispositivo autorizado:** descobrir o notebook online pelo Desktop Commander.
2. **Conferir o ambiente:** horário, sistema, sessão Cinnamon/X11, uptime, memória e processos.
3. **Listar services do usuário:** comandos systemctl --user list-units, list-unit-files e show, com PID, status e reinícios.
4. **Listar serviços de sistema:** consultas equivalentes a SentinelX, áudio, rede, impressão e rotinas do SO.
5. **Investigar autostart:** verificar arquivos .desktop da sessão e unidades habilitadas do systemd.
6. **Relacionar consumo:** ps, free, uptime e psutil. A intenção era separar o custo do Gateway e o custo do computador inteiro.
7. **Mapear portas:** ss e conexões locais para diferenciar serviço ativo de endpoint realmente acessível.
8. **Ler registros de falha:** journalctl do Gateway e da unidade de recuperação, sem mudar configurações.
9. **Registrar baseline:** estados e screenshot anterior à reinicialização, com hash de evidência.

Descobertas do inventário

- O Gateway usa um serviço do systemd do usuário; a UI do navegador não é requisito para ele existir.
- O subsistema de recuperação do navegador possui uma unidade **separada**, com possibilidade de falha própria.
- Desktop Commander e SentinelX são integrações distintas; foram apenas monitoradas.
- A máquina possui aproximadamente **7,7 GiB de RAM**, com carga variável no momento do boot.
- O DSH não participa do mecanismo de monitoramento, apesar de estar instalado.
- A sessão Cinnamon/X11 permite testar a UI visualmente no Chrome sem abrir portas públicas.

Decisão de engenharia: observar antes de corrigir. Alterar primeiro a unidade de recuperação poderia mascarar um problema de ordenação ou prontidão do Gateway.

04 — Requisitos e justificativas de tecnologia

Requisitos de produto

A solução deveria apresentar CPU/RAM, histórico gráfico, estado do processo Gateway e filhos, portas TCP locais, inventário systemd, logs recentes, alertas, atalho na área de trabalho, funcionamento no reboot e interface responsiva.

Requisitos não funcionais

Sem necessidade de nuvem para coletar métricas; sem alterar agentes; dependências reduzidas; CPU e memória baixas no próprio painel; dados organizados; segurança por padrão via loopback; compatibilidade com Linux Mint.

DECISÃO	POR QUÊ	LIMITE CONHECIDO
Python + biblioteca HTTP padrão	Servidor leve sem framework obrigatório	Sem funções prontas de frameworks maiores
psutil	CPU, RAM, filhos de processo, uptime	Soma de RSS pode duplicar páginas compartilhadas
systemctl / journalctl	Fontes do Linux para status e eventos	Leitura depende das permissões disponíveis
TCP local	Verificar aceitação de conexão por porta	TCP ativo não equivale a serviço funcional de ponta a ponta
SQLite com WAL	Histórico local com leitura concorrente	Coletor controla limpeza e retenção
HTML/CSS/JavaScript	Front-end sem build complexo ou CDN	Componentes gráficos são mantidos manualmente
Canvas	Gráfico sem biblioteca externa	Escalas, ajustes e amostragem dependem do código
systemd de usuário	Inicialização e restart sob supervisão	Precisa de gerenciador do usuário ativo
HTTP em loopback	Reduzir exposição de dados operacionais	Acesso externo direto não é permitido
PDF HTTPS separado	Compartilhar documentação sem expor endpoints	PDF público pode ser lido por qualquer pessoa com o link

Inventário de artefatos de código

O projeto possui cinco arquivos principais: server.py (coleta e API), index.html (estrutura da interface), style.css (layout e responsividade), app.js (renderização e polling), README.md (operação). O banco fica na pasta data/ e a configuração de inicialização é uma unidade mcf-openclaw-observer.service sob systemd.

A distribuição do relatório utiliza uma cópia pública revisada. Caminhos absolutos privados, senhas, tokens, endereços internos e credenciais foram suprimidos; os conceitos da arquitetura permanecem descritos.

05 — Construção do backend: responsabilidades concretas

O backend é um processo Python. Ele possui **dois fluxos cooperantes**: um loop em thread que coleta telemetria e um servidor HTTP que responde às páginas e aos endpoints sem repetir todas as consultas do SO para cada visitante.

Passo a passo do loop de observação

1. O sistema consulta as unidades relevantes no systemd e extrai suas propriedades estruturadas: ActiveState, SubState, MainPID, NRestarts, ExecMainStatus e tempo desde a ativação.
2. A partir do PID do Gateway, identifica todos os filhos recursivos da árvore, quando existem.
3. Utiliza psutil para amostrar CPU e memória de cada processo. Casos em que um processo desaparece durante a leitura são tolerados.
4. Lê CPU geral e consumo agregado de memória física da máquina.
5. Testa a capacidade de conexão TCP em três portas conhecidas: uma do Gateway, uma do navegador gerenciado e outra de uma instância auxiliar de Chrome.
6. Produz alertas para falhas de serviço, porta não acessível e consumo acima de limiares.
7. Atualiza um snapshot estruturado compartilhado com a API HTTP, dentro de um lock de sincronização.
8. Persiste amostra em SQLite, em uma transação local.
9. Aguarda cinco segundos e recomeça. Esse intervalo é aproximado porque coleta e I/O também consomem tempo.

Funções verificadas no arquivo Python real

IDENTIFICADOR	FUNÇÃO DE ENGENHARIA
run	Executa subprocesso com timeout e captura de stdout
systemd_info	Transforma propriedades nativas do systemd em objeto de dados
process_tree	Encontra PID principal e descendentes
processes_snapshot	Soma CPU/RAM e produz ranking dos processos relacionados
is_listening	Tenta conexão TCP local com timeout curto
boot_uptime	Calcula uptime do kernel
collect	Combina estados, recursos, sockets e alertas
db_open	Configura e abre o banco SQLite
tick	Realiza coleta recorrente e gravação
history	Seleciona e reduz pontos para o gráfico
logs	Recupera eventos do journal e oculta linhas sensíveis
Handler.do_GET	Serve páginas estáticas, estado, histórico e logs
Handler.do_POST	Rejeita métodos que poderiam modificar estado
main	Prepara banco, inicia coletor e servidor HTTP

Por que manter um snapshot em memória?

Se cada recarregamento da página executasse dezenas de processos systemctl e journalctl, abrir duas abas poderia duplicar desnecessariamente o custo. O coletor atualiza os dados uma vez; as telas apenas os leem. É o padrão **producer-consumer**: o coletor produz as medições e a API as entrega aos consumidores.

O objeto de estado é protegido por threading.RLock, um mecanismo de sincronização. O snapshot é atualizado em seção crítica curta. O próprio coletor tem tratamento de exceção, de modo que uma falha pontual não deveria encerrar permanentemente o serviço; a ocorrência é registrada como tipo de erro de coleta.

Modelo de API de alto nível



Os endpoints não oferecem comandos de restart, shell, criação de agentes, navegação autenticada ou alteração de chaves. Esse escopo reduz a superfície de risco.

06 — Matemática de CPU, memória e disponibilidade

CPU do Gateway: é a soma de CPU (%) observada nos processos da árvore do Gateway. Sob Linux, 100% representa aproximadamente um núcleo lógico totalmente ocupado, e valores de 200% ou 280% são possíveis em máquinas com vários núcleos. Isso não significa “280% de todos os processadores”.

RAM do Gateway: soma os campos RSS (Resident Set Size), ou seja, páginas de memória residentes por processo. Páginas compartilhadas entre processos podem aparecer contabilizadas em mais de um RSS. Consequentemente, o número é adequado para **tendência e alerta**, mas não é uma medição exata de memória física exclusiva.

CPU do notebook: uso percentual global retornado por psutil.cpu_percent, com normalização distinta do somatório de CPU do Gateway.

RAM do notebook: porcentagem e quantidade em GiB lidas por psutil.virtual_memory. A RAM “usada” não implica que toda memória remanescente esteja irrecuperável; caches do Linux fazem parte da dinâmica normal.

Processos filhos: a árvore pode mudar de uma amostra para outra. Se um processo é encerrado enquanto está sendo consultado, a leitura dessa instância pode ser pulada sem invalidar a coleta inteira.

Uptime: tempo desde o início do kernel. Um processo Gateway reiniciado pode ter idade menor que o uptime total, razão pela qual o painel distingue data de boot e PID.

Disponibilidade TCP: conexão curta e local confirma apenas aceitação de socket. Não executa login, scraping, sessão CDP completa ou inferência de modelo. Portanto, uma porta aberta é uma condição necessária para certas ações, mas não suficiente para afirmar saúde integral.

Thresholds: regras iniciais alertam para CPU somada a partir de 100% e memória somada acima de 1.536 MiB, além de falhas em serviços e portas. Esses valores foram escolhidos como referência operacional, não como diagnóstico científico individualizado para cada configuração.

Limitações declaradas

- Uma leitura de CPU é uma estimativa por intervalo de amostragem, não uma medição absoluta de trabalho.
- O somatório RSS pode superestimar páginas compartilhadas.
- A coleta a cada cinco segundos pode perder picos curtos.
- O histórico é reduzido em alguns intervalos para cerca de 400 pontos, o que pode suavizar visualmente picos estreitos.
- Uma falha de leitura systemd pode aparecer como estado desconhecido, em vez de comprovar ausência do serviço.
- O teste de socket não é um teste end-to-end do navegador.

07 — Persistência: estrutura da base SQLite

O MCF Observer mantém telemetria local em um banco SQLite. Não exige Postgres, Redis, Supabase, conexão com nuvem ou um servidor de métricas adicional.

Esquema conceitual

COLUNA	TIPO	INTERPRETAÇÃO
ts	INTEGER PRIMARY KEY	Tempo Unix da amostra
cpu	REAL	CPU somada da árvore Gateway
ram	REAL	RSS somado, em MiB
host_cpu	REAL	CPU normalizada do notebook
host_ram	REAL	Porcentagem de RAM do notebook
online	INTEGER	1 quando Gateway ativo e porta do Gateway acessível; 0 caso contrário

Decisão de concorrência

SQLite utiliza **WAL (Write-Ahead Logging)**, um arquivo auxiliar de log de escrita que permite que consultas leiam enquanto outras transações são gravadas. O coletor mantém conexão própria; solicitações de histórico criam conexões de consulta distintas. Os commits finalizam cada inserção.

Retenção e economia de dados

O objetivo de retenção é um histórico de até 24 horas, com descarte periódico de linhas antigas pelo coletor. A coleta nominal de cinco segundos produz potencialmente até **17.280 amostras em 24 horas** sem pausas. A consulta limita a série exibida a cerca de 400 pontos. Uma variável de teto de linhas existe no código, mas não é, por si só, prova de um mecanismo adicional de limpeza; a regra efetiva é o DELETE por idade disparado pelo loop.

Privacidade

O banco contém números de CPU, RAM, timestamps e estado binário. Não precisa guardar mensagens dos agentes, prompts do usuário, chaves, conteúdos de conversas ou URLs visitadas. Foi observada permissão de acesso restrita ao usuário para o arquivo do banco. A cópia pública do relatório **não inclui esse banco**.

08 — Engenharia da interface: experiência e arquitetura visual

O front-end foi escrito sem React, Vite ou outra cadeia de compilação. A arquitetura visual tem uma **barra lateral persistente**, três visões e uma atualização periódica.

Estrutura das três páginas

Visão geral: indicadores principais (CPU Gateway, RAM Gateway, CPU do notebook, RAM do notebook), faixas de estado, gráfico de histórico, disponibilidade de portas, alertas ativos e tabela dos processos associados.

Serviços & Boot: unidades systemd observadas separadas por categorias (core, integração e sistema). Mostra o estado atual, a habilitação e o PID, quando disponível. Explica a diferença entre “habilitado” e “ativo”.

Logs & Diagnóstico: troca de fonte dos registros, atualização manual, eventos recentes e cartões de alerta. As linhas com palavras-chave de possíveis segredos são ocultadas preventivamente.

Ciclo de renderização

O app.js consulta /api/state e atualiza os elementos do DOM por suas IDs. Os painéis são renderizados a partir de JSON. Para listas, o JavaScript cria nós DOM e define textContent em vez de inserir HTML arbitrário; isso evita interpretar logs como código executável.

O gráfico utiliza canvas com escala própria para CPU e RAM. Ajusta dimensões à largura disponível, exibe grade, curvas e legendas, e responde a resize. Os períodos 5 minutos, 15 minutos e 1 hora estão disponíveis na UI; o backend suporta também janelas mais longas.

Atualização e tolerância à falha

- Métricas e estado: atualização a cada cinco segundos.
- Logs: atualização enquanto a aba de diagnóstico está aberta, aproximadamente a cada 30 segundos.
- Horário visual: relógio local atualizado a cada segundo.
- Falhas HTTP: UI troca o indicador de conexão; tenta novamente no ciclo seguinte.
- Consulta da API: timeout interno no JavaScript para não ficar aguardando indefinidamente.
- Responsividade: a barra lateral compacta muda de layout em telas menores.

Por que validação visual é necessária?

Uma API pode responder HTTP 200, mas a tela apresentar layout quebrado, gráfico sem dados ou componentes escondidos. Por isso, além de testar JSON, executamos Chrome headless contra o painel, geramos screenshots, verificamos o DOM e inspecionamos visualmente as imagens.

Screenshots do trabalho

As quatro imagens documentais deste PDF representam **uma fotografia anterior ao reboot**, a **visão geral depois do boot**, a **lista de serviços** e a **página de diagnóstico**. Para divulgação pública, áreas de logs potencialmente identificáveis foram redigidas na cópia publicada; os arquivos originais permanecem exclusivamente no notebook.

09 — Implantação pelo systemd e atalho no desktop

O painel foi transformado em serviço gerenciado pelo **systemd do usuário**, em vez de depender de um terminal aberto. O processo principal do serviço é o servidor Python, que inicia também o coletor periódico.

PROPRIEDADE DO SERVIÇO	VALOR INSTALADO	SIGNIFICADO
Description	MCF OpenClaw Observer	Identificação da unidade
Type	simple	Gerenciador acompanha o processo principal
ExecStart	Python executando o servidor	Comando de inicialização
WorkingDirectory	Pasta do aplicativo	Diretório de trabalho
Restart	on-failure	Reinicia o painel se este falhar
RestartSec	4	Aguarda quatro segundos
NoNewPrivileges	true	Impede aquisição posterior de privilégios
UMask	0077	Arquivos novos restritos ao usuário
WantedBy	default.target	Vincula ao início do gerenciador do usuário

Sequência aplicada: criação do arquivo de unidade, validação de sintaxe, daemon-reload, enable --now, consulta do estado, testes de API e validação depois de reboot. O resultado pós-reboot confirmou que o MCF Observer iniciou novamente sem intervenção manual.

Foi instalado também um arquivo .desktop, registrado no menu e colocado como atalho na área de trabalho. Sua função é abrir a página local no navegador. O serviço e o atalho são independentes: fechar o navegador não encerra a coleta.

Observabilidade da própria aplicação: a rota /health mostra que o servidor HTTP está respondendo. Não significa que o Gateway observado esteja saudável; essa distinção aparece nas demais rotas.

10 — Segurança, isolamento e limitações operacionais

A segurança foi desenhada para um **painel somente de leitura na própria máquina**. A versão 1.0 não tem sistema próprio de login, porque não deveria estar exposta à rede: a interface HTTP escuta apenas no loopback.

CONTROLE	EVIDÊNCIA TÉCNICA	LIMITAÇÃO
Escuta local	Socket vinculado a 127.0.0.1	Software da mesma conta pode acessar
Mutação desabilitada	POST recebeu HTTP 405	Impede ações via interface; não é antivírus
Host validado	Cabeçalho de host inesperado recebeu HTTP 403	Não substitui autenticação
Política CSP	Scripts, estilos e conexões limitados à própria origem	Não protege extensões do próprio navegador
Anti-framing e nosniff	Cabeçalhos X-Frame-Options e X-Content-Type-Options	Defesa adicional de browser
Logs filtrados	Linhas potencialmente sensíveis são ocultadas	Pode esconder informações úteis ao diagnóstico
SQLite com modo restrito	Banco observado em modo 0600	Não há criptografia adicional do banco
Sem operação em agentes	Nenhuma rota HTTP de restart ou execução de comandos	Observador não remedia incidentes
Documento público estático	Somente PDF selecionado para compartilhamento	Quem conhece o link pode lê-lo

Atenção: TLS no link público protege o transporte, não transforma o PDF em documento privado. A cópia publicada não contém arquivos originais de log, banco SQLite, registros de conversas ou credenciais de serviços. A versão completa permanece no notebook.

Os logs usam filtro por palavras-chave, o que reduz risco de vazar dados na interface, mas não equivale a uma solução formal de prevenção de vazamentos. Portanto, a cópia divulgada é preparada separadamente das fontes operacionais.

11 — Plano de testes em camadas e critérios de aceitação

A engenharia foi validada em etapas. Cada verificação tem um **escopo limitado**: sucesso de uma rota HTTP não prova automaticamente que todos os agentes do OpenClaw funcionam.

CAMADA	COMO FOI EXECUTADA	RESULTADO
Sintaxe Python	Compilação do arquivo server.py	PASS
Sintaxe JavaScript	Validação de app.js via Node.js	PASS
Serviço de usuário	Consulta de unidade ativa pelo systemd	PASS
Socket do painel	ss e curl contra o endereço local	PASS
API de estado	GET /api/state e validação de campos JSON	PASS
API de histórico	GET /api/history e contagem de amostras	PASS
API de logs	GET /api/logs para fontes permitidas	PASS
Rotas negativas	POST negado, Host inesperado e fonte inválida	PASS
Persistência	Consulta SQLite, histórico e permissões	PASS
UI renderizada	Chrome headless, screenshot e leitura de DOM	PASS
Inicialização depois de reboot	Novo boot ID, reconexão e estado do Observer	PASS
Gateway depois do reboot	Estado, socket e mensagem READY no journal	PASS após retry
Navegador gerenciado	Serviço recuperador e porta CDP dedicada	FAIL
Documento original de evidências	PDF inspecionado com utilitários de PDF	PASS

Dupla validação: API mais interface

Foi empregada uma abordagem em dois níveis. Primeiro validamos JSON e códigos HTTP; depois abrimos a UI em Chrome headless e geramos capturas reais da renderização. Isso evita uma falsa conclusão em que o backend funciona, mas os cards, botões ou gráficos não aparecem corretamente.

Testes negativos

Uma API segura precisa recusar operações fora de escopo. Foram verificados explicitamente os retornos 405 para POST, 403 para Host inesperado e 400 para uma fonte de logs que não constava na lista permitida. O fato de os testes falharem **como esperado** é um resultado positivo de segurança.

12 — Experimento controlado de reinicialização

O roteiro foi: salvar baseline, registrar captura do painel antes do reboot, reiniciar a máquina, reconectar, confirmar um novo boot ID, esperar o Gateway, observar os estados dos serviços, gerar três novas capturas e consolidar o relatório.

HORÁRIO LOCAL APROXIMADO	EVENTO DOCUMENTADO
05:08	Estado pré-reboot e screenshot registrados
05:08:53	Novo kernel iniciou
05:09:00	Recuperação do navegador iniciou
05:09:13	Primeira execução do Gateway terminou com código 1
05:09:18	systemd iniciou nova tentativa do Gateway
05:09:40	Observer ativo; recuperação do navegador falhou com código 2
05:10:05	Gateway passou a registrar HTTP acessível
05:10:14	Gateway registrou READY
05:11	Capturas das três visões pós-reboot geradas
05:12–05:15	API, logs, screenshot e PDF auditados

O boot ID anterior e posterior foram diferentes, confirmando a reinicialização do **computador inteiro**, e não apenas do aplicativo. O Gateway passou por uma tentativa inicial malsucedida e se recuperou depois, pela supervisão do systemd.

Diferença importante entre serviço iniciado e serviço pronto

Na unidade do navegador existe uma dependência do tipo After sobre o Gateway. Essa opção controla **ordenação**, mas não garante que a API do Gateway completou todas as fases internas. A rotina do navegador começou antes do READY, o que sustenta a investigação de uma possível *race condition* (corrida entre inicializações).

Esta é uma hipótese **plausível, não comprovada**. Pode haver outros bloqueios de perfil, CLI ou contexto gráfico. Nenhum mecanismo de recovery foi alterado sem evidência suficiente.

Amostras de recursos

Antes da reinicialização foi observada RAM do grupo Gateway em torno de 2 GiB em algumas amostras. Após reboot, a captura registrou valor diferente, em torno de 1,5 GiB, com variações de CPU durante a inicialização. Não se trata de média mensal nem previsão de consumo constante.

13 — Falhas que o relatório preserva sem corrigir

Recuperação do navegador: o serviço terminou com status 2. A interface CDP gerenciada permaneceu indisponível. O código do recuperador testa status, tenta iniciar normalmente, tenta modo headless e verifica novamente. A falha ainda exige testes diretos para discriminar entre prontidão do Gateway, parâmetros CLI e condições do browser.

Gateway: a primeira partida após reboot terminou com código 1 e uma nova execução do systemd conseguiu o estado READY. Isso comprova capacidade de recuperar o serviço, mas não a funcionalidade de todos os agentes.

Skill inválida do browser operator: os logs reportaram que uma skill foi descartada por não declarar o campo técnico de metadados description. O problema foi registrado; não houve edição do conteúdo dessa skill.

casper-md5check: o serviço de verificação da mídia de instalação também estava falhando. É um achado do sistema operacional sem vínculo causal demonstrado com a aplicação observada.

Performance: o uso de RAM merece acompanhamento. Os limites de alerta iniciais são deliberadamente simples e podem necessitar calibração por séries históricas mais extensas.

Resultado do experimento

PASS: reboot comprovado, reconexão, observador persistente, API, UI, Gateway após retry, gravação de dados.
FAIL: recuperação do navegador e porta gerenciada correspondente.
PENDENTE: comprovar causa-raiz e aplicar correções em missão futura, com rollback documentado.

14 — Registro de ferramentas utilizadas pela assistência

Esta seção identifica **as ferramentas realmente acionadas para implementar e verificar o MCF Observer**. O registro é por operação técnica; não expõe argumentos que contenham identificadores internos ou trechos de sessão privados.

FERRAMENTA	OPERAÇÕES COMPROVADAS NESTA MISSÃO	MOTIVO
Remote Desktop Commander — list_devices	Descobrir notebook autorizado e disponibilidade	Não presumir acesso ativo
Remote Desktop Commander — start_process	Comandos de shell, Python, Node, curl, systemctl, journalctl, ss, SQLite e Chrome	Executar trabalho no notebook real
Remote Desktop Commander — read_process_output	Recuperar saída de execução assíncrona	Confirmar término, erro e resultado
Remote Desktop Commander — write_file	Gravar server.py, index.html, CSS, JavaScript e documentação	Criar componentes persistentes do painel
Remote Desktop Commander — read_file	Ler screenshots PNG produzidos no notebook	Inspeção visual genuína
GitHub — get_repo	Consultar repositório oficial do MCF	Recuperar fonte canônica do projeto
GitHub — fetch_file	Ler contrato do papel MESTRE	Delimitar orquestração e responsabilidades
Ferramentas Linux (via shell remoto)	systemctl, journalctl, ss, ps, free, uptime, loginctl, xinput	Diagnóstico nativo do ambiente
Navegador Chrome headless (via shell)	Abertura de páginas, screenshot, dump-dom e impressão para PDF	Validar layout e registrar evidência
Ferramentas de PDF (via shell)	pdfinfo, pdftotext, pdfimages e pdftoppm	Conferir páginas, texto, imagens e renderização
Vercel (publicação separada)	Implantação de arquivos estáticos e consulta de status	Gerar um URL HTTPS público para o PDF revisado

Não foi executada uma sequência autônoma dentro do OpenClaw por outro agente para construir este painel. O acesso remoto não se confunde com um “agente inteligente novo”; é uma capacidade de executar instruções autorizadas na máquina.

Passos reais representativos de chamadas

1. Consultar estado: systemctl --user show openclaw-gateway.service.
2. Ler journal: journalctl --user -u openclaw-gateway.service, limitado a últimas linhas e com cuidados de filtragem.
3. Verificar sockets: ss -lntp.
4. Validar a API: curl no endereço HTTP loopback /api/state.
5. Validar JavaScript: node --check app.js.
6. Validar Python: python3 -m py_compile server.py.
7. Instalar serviço: systemctl --user daemon-reload; enable --now.
8. Testar método proibido: requisição POST na API, com retorno 405 esperado.
9. Testar origem indesejada: cabeçalho Host inválido, com retorno 403 esperado.
10. Testar dados: consulta a samples no SQLite e leitura do histórico.
11. Gerar screenshot: google-chrome em modo headless com screenshot e dump-dom.
12. Reiniciar notebook: solicitação systemctl reboot após salvar baseline.
13. Provar novo boot: comparar identificadores de boot anteriores e posteriores.
14. Validar PDF: pdfinfo, pdftotext, pdfimages, pdftoppm e inspeção visual das páginas.
15. Publicar cópia revisada: arquivos estáticos em serviço HTTPS externo, sem abrir a rede do notebook.

Essas chamadas são registradas como **descrição técnica auditável**, não como transcrição integral dos payloads brutos. A transcrição integral de comandos de máquina pode conter caminhos de usuários, contexto de autenticação e outros dados inadequados para divulgação pública.

15 — Agentes e LLMs: atribuição factual do trabalho

O termo **MESTRE** é o papel de orquestração adotado nesta conversa. Não significa automaticamente que o processo do agente mcf-mestre dentro do OpenClaw tenha recebido uma missão.

Quem de fato trabalhou na implementação do painel?

ENTIDADE	PAPEL NESTE TRABALHO	EVIDÊNCIA DE UTILIZAÇÃO
GPT-6 (assistente da conversa)	Especificação, decisões, código, testes e relatório	Geração das instruções e chamadas de ferramentas
Remote Desktop Commander	Execução remota e inspeção	Chamadas diretas à máquina autorizada
OpenClaw Gateway	**Objeto observado**, não executor do painel	systemd, psutil, sockets, journal
Serviço de recuperação do navegador	**Objeto diagnosticado**	Logs e código de retorno
Google Chrome headless	Motor de renderização, **não LLM**	Screenshots e PDF
Agentes internos OpenClaw	Existiam na instalação; **não convocados** para este trabalho	Não há execução delegada a eles nesta missão
Modelos NVIDIA NIM	Configuração identificada em logs de outro subsistema	Nenhuma inferência NIM solicitada para criar ou validar o painel

Modelo de linguagem: o que foi observado e o que não foi

Modelo que produziu a engenharia em chat: GPT-6. A ferramenta remota executou comandos concretos e programas locais, sem necessariamente usar um modelo de IA próprio para cada chamada.

Modelo identificado na inicialização do OpenClaw: a linha de log observada para o Gateway indicou um identificador de modelo Nemotron Lightning via NVIDIA, com configuração de esforço de raciocínio. Isso comprova **qual modelo estava anunciado/configurado**, não que tenha sido chamado para programar o painel ou que uma inferência bem-sucedida ocorreu no teste de boot.

Outros modelos NVIDIA testados anteriormente no DSH: pertencem a uma missão anterior e não foram usados para produzir esta aplicação. Não transferimos seus resultados automaticamente para o MCF Observer.

Testes de fumaça de agentes anteriores: em outra atividade do histórico, foram enviados pedidos de resposta a 41 agentes OpenClaw. Isso também **não constitui convocação desses 41 agentes** para esta implementação.

O que não foi feito

Não houve criação de um novo agente autônomo, treinamento de pesos, modificação de LLM, execução de um enxame de agentes para desenvolver o painel, alteração de modelos padrão nem autorização de agentes para navegar em contas durante este trabalho.

Regra de atribuição: atribuir uma mudança ao componente que realmente a fez; não presumir que um processo em background “trabalhou conosco” apenas porque estava em execução.

16 — Controle de mudanças, governança e reversibilidade

Escopo de escrita executado

O trabalho introduziu arquivos próprios do aplicativo de observabilidade, sua unidade systemd de usuário, o lançador da área de trabalho e relatórios com capturas. A única alteração posterior no aplicativo foi acrescentar rotas locais para baixar o relatório e um botão de acesso ao PDF no cabeçalho.

Escopo preservado

Não houve alteração do script do recuperador do navegador; não houve edição da configuração de agentes, modelos ou credenciais do OpenClaw; não houve abertura de porta pública do notebook; não houve mudança no firewall; nenhum serviço do SentinelX foi desativado como parte do experimento.

Como reverter o observador sem mexer no OpenClaw

A reversão operacional deve ser conduzida em etapa autorizada separada: parar e desabilitar a unidade do Observer; remover seu atalho; arquivar ou excluir apenas a pasta do próprio Observer após guardar dados relevantes. O Gateway e o recuperador do navegador são serviços diferentes, que **não devem ser removidos** nessa reversão.

Pontos de restauração

O código do observador possui cópias das versões anteriores a algumas mudanças pequenas, como o acréscimo da rota do relatório. O relatório pós-reboot guarda o baseline anterior, o estado posterior e capturas originais. O dossiê público é uma nova cópia estática e sua exclusão na hospedagem não exige alterar o computador.

17 — Engenharia da publicação pública

O relatório disponível no notebook originalmente foi servido por uma rota HTTP acessível **somente localmente**. Esse endereço não é um link público adequado para o telefone quando o usuário está fora de casa.

Para cumprir a entrega externa, foi escolhida uma **publicação estática por HTTPS** em uma plataforma de hospedagem, contendo apenas o documento revisado e eventualmente uma página de apresentação. Não se publica o código do Gateway, banco SQLite, API interna, scripts com credenciais ou logs brutos.

Diferença entre as URLs

URL	FINALIDADE	PÚBLICO?
HTTP local do Observer	Métricas ao vivo do notebook	Não; restrita ao próprio computador
HTTP local /relatorio	PDF detalhado armazenado no notebook	Não; também restrita ao computador
HTTPS público /relatorio.pdf	Cópia estática compartilhável	Sim; legível por quem possuir a URL

Segurança e privacidade do arquivo público

A publicação usa criptografia TLS para a transmissão. Isso não significa controle de acesso por senha. As telas do painel podem revelar nomes de serviços, estados e títulos; as regiões de logs e identificadores privados foram tratadas na **cópia de divulgação**, mantendo as imagens originais no ambiente local.

Rastreabilidade

O relatório relaciona seus achados a quatro fontes de evidência: código do Observer, serviço systemd, snapshots e screenshots pós-reboot, e mensagens do journal. No notebook, as imagens originais e arquivos JSON permitem revisitar as provas. A cópia pública apresenta o que pode ser divulgado, sem anexar dados operacionais brutos.

18 — Próximo ciclo de engenharia (proposto, ainda não executado)

A ordem de correção deve respeitar a evidência: o Gateway se recuperou, o navegador não. O trabalho seguinte deve isolar a origem exata da falha antes de modificar regras de inicialização.

FASE FUTURA	EXPERIMENTO PROPOSTO	ACEITE NECESSÁRIO
A — Baseline	Congelar estado da unidade e script atuais	Backup e hashes registrados
B — Ready check	Confirmar condição operacional real do Gateway após boot	Readiness comprovado por resposta da API
C — CLI browser	Rodar status/start com parâmetros documentados, sem expor credenciais	Evidência de saída e retorno
D — Dependência	Se confirmado, introduzir espera limitada por readiness	Não criar loop infinito nem bloquear login
E — CDP	Verificar endpoint gerenciado de navegador	Porta esperada e protocolo funcional
F — Navegação	Exercício não autenticado e reversível	Página de teste carregada e screenshot obtido
G — Regresso	Reiniciar o notebook e repetir auditoria	Sem nova falha de recuperação
H — Governança	Registrar mudança, diff, logs e rollback	Relatório de testes e aceitação pelo proprietário

Essa tabela é planejamento. **Nenhuma dessas alterações foi aplicada nesta missão.**

Recomendações de evolução do Observer

- Criar métricas de *readiness* do Gateway, distintas de porta aberta.
- Separar processos principais de sidecars para evitar interpretação errada de RAM.
- Melhorar o filtro de logs com sanitização estruturada, reduzindo ocultação excessiva.
- Registrar eventos de transição de estado, não apenas amostras periódicas.
- Criar histórico de alertas e limiares calibrados após dias de medição.
- Desenvolver testes automatizados unitários e integração de ponta a ponta.
- Acrescentar autenticação forte caso o painel precise ser acessado remotamente, em vez de expor o HTTP local.

19 — Glossário técnico em linguagem natural

TERMO	EXPLICAÇÃO PRÁTICA
Agente	Programa ou sistema que pode receber tarefas e decidir passos para executá-las. Não equivale automaticamente a um processo do sistema operacional.
API	Interface de dados pela qual dois programas se comunicam.
Backend	Parte da aplicação que coleta, calcula e entrega informações.
Front-end	Camada visual com a qual o operador interage no navegador.
Gateway	Componente central do OpenClaw responsável pela comunicação e coordenação de serviços.
systemd	Supervisor do Linux que inicia, acompanha e reinicia serviços.
Unit / serviço	Definição administrada pelo systemd, com estados e dependências.
One-shot	Serviço planejado para executar uma tarefa e terminar.
After	Diretiva de ordenação do systemd; não equivale a confirmação de prontidão funcional.
Readiness	Evidência de que a aplicação terminou o boot e aceita a operação necessária.
Race condition	Problema cujo resultado depende da ordem e do timing entre duas tarefas.
Processo filho	Programa iniciado por outro processo, incluído na árvore do processo pai.
PID	Número de identificação temporário de um processo.
psutil	Biblioteca Python para ler CPU, RAM, processos e informações do sistema.
RSS	Memória residente registrada para um processo; páginas compartilhadas podem ser contabilizadas em mais de um processo.
CPU acima de 100%	Em Linux, pode indicar uso de mais de um núcleo lógico por uma árvore de processos.
Polling	Consultas periódicas, como verificar métricas a cada cinco segundos.
Snapshot	Fotografia estruturada do estado do sistema num instante.
Thread	Fluxo de execução dentro do mesmo processo.
Lock	Mecanismo que coordena acesso simultâneo ao mesmo dado.
SQLite	Banco de dados local em arquivo, sem servidor separado.
WAL	Modo SQLite de registro antecipado de escrita, favorável à concorrência de leitores.
Endpoint	Caminho da API que serve um tipo de informação.
HTTP 200	Requisição concluída com sucesso.
HTTP 403	Acesso rejeitado.
HTTP 405	Método HTTP não permitido.
HTTPS / TLS	Conexão criptografada entre cliente e servidor remoto; não torna um documento automaticamente privado.
Loopback	Endereço de rede usado apenas dentro da própria máquina.
TCP	Protocolo de transporte usado, entre outras coisas, para comunicação com serviços web.
CDP	Chrome DevTools Protocol; protocolo de automação do navegador.
Headless	Navegador executado sem janela interativa visível, útil para testes e screenshots.
DOM	Estrutura interna dos elementos HTML que o navegador renderiza.
Canvas	Área HTML desenhada por JavaScript, usada neste painel para o gráfico.

TERMO	EXPLICAÇÃO PRÁTICA
CSP	Política que restringe de onde a página pode carregar scripts e outros recursos.
Journal / journalctl	Sistema e comando de consulta de logs estruturados do Linux.
Rollback	Voltar ao estado anterior conhecido após uma alteração.
Smoke test	Teste rápido que verifica se os componentes essenciais respondem.
End-to-end	Teste do fluxo completo, da entrada até o resultado final.
LLM	Modelo de linguagem capaz de interpretar e gerar texto e instruções.
Orquestração	Processo de coordenar tarefas, ferramentas e agentes segundo critérios e restrições.

20 — Conclusão técnica e fontes de verdade

A arquitetura estabeleceu um limite claro entre **observação** e **controle operacional**. O MCF Observer monitora sem possuir ferramentas de escrita sobre o OpenClaw. Isso ajudou a preservar os sintomas importantes após reinicialização e forneceu evidência para o próximo trabalho.

O sucesso do painel não apaga as falhas do navegador. O Gateway voltou, o Observer voltou e as interfaces foram renderizadas. A recuperação do browser não voltou, e a possível corrida de inicialização segue como hipótese. Essa distinção é o principal resultado de engenharia da fase atual.

Documentos-fonte mantidos localmente

- Código original do Observer: servidor Python, HTML, CSS e JavaScript.
- Configuração ativa da unidade mcf-openclaw-observer.service no systemd do usuário.
- Estados antes_reinicio.json e apos_reinicio.json, guardados com o relatório anterior.
- Quatro screenshots PNG genuínos, coletados antes e depois do boot.
- Relatório anterior de oito páginas, incluindo as capturas originais.
- Históricos de recursos do banco SQLite e registros do journal do notebook.
- Contrato do papel MESTRE consultado no repositório oficial do MCF.

Observação de divulgação

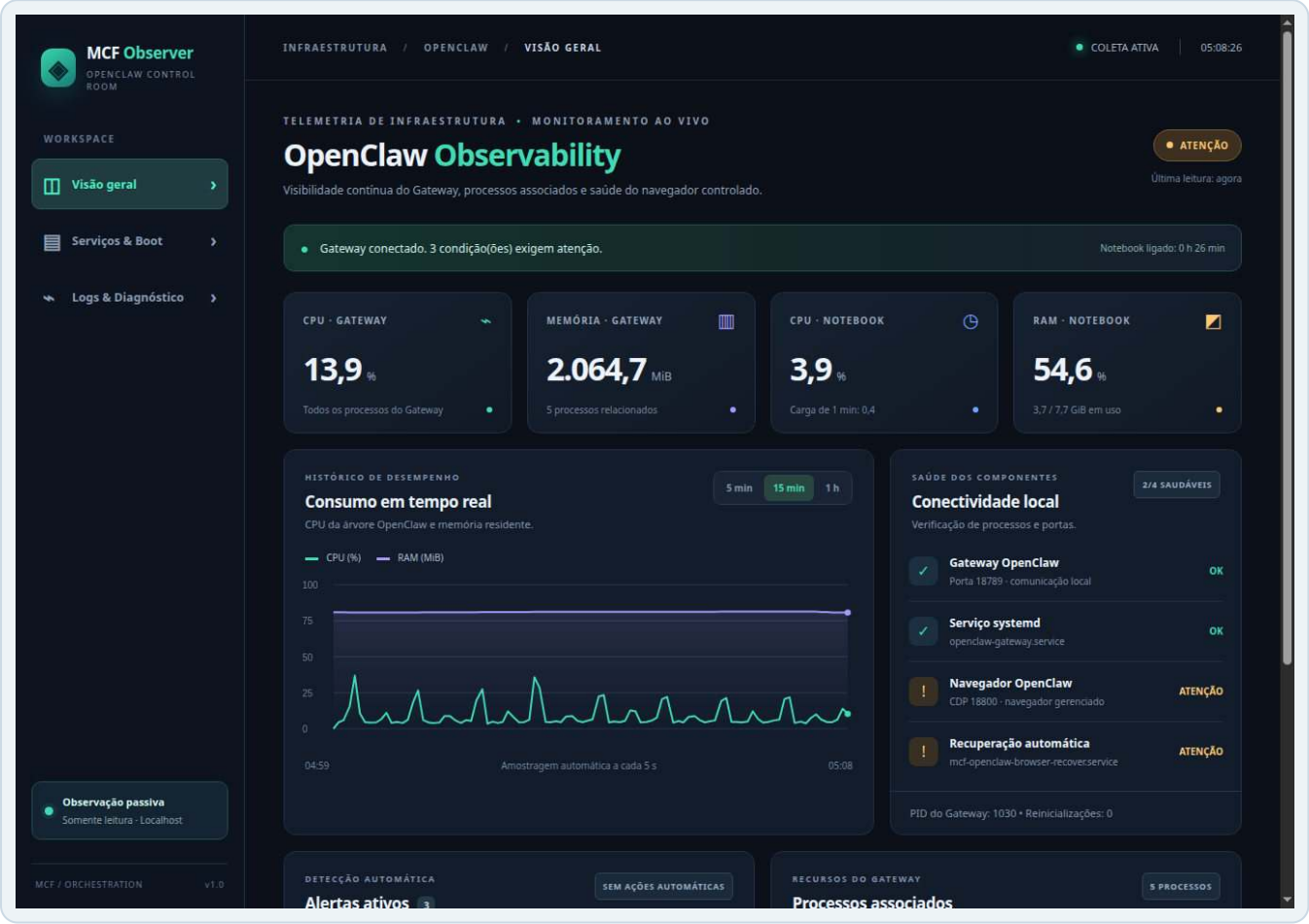
A versão pública aqui entregue explica o projeto de modo verificável, mas não é um *dump* das contas, logs brutos, APIs privadas ou chaves. Não foram publicados arquivos que permitam controlar o notebook. Para auditoria detalhada de logs internos, deve-se utilizar a cópia que permanece sob controle do proprietário.

Situação de encerramento deste documento: arquitetura descrita, relatórios e screenshots preservados, sistema monitorado não modificado e subsistema do navegador aguardando missão específica de correção.

EVIDÊNCIA VISUAL 01/04 · ORIGEM: CAPTURA REAL

ANTES DA REINICIALIZAÇÃO

Estado do painel registrado antes do experimento de reboot.

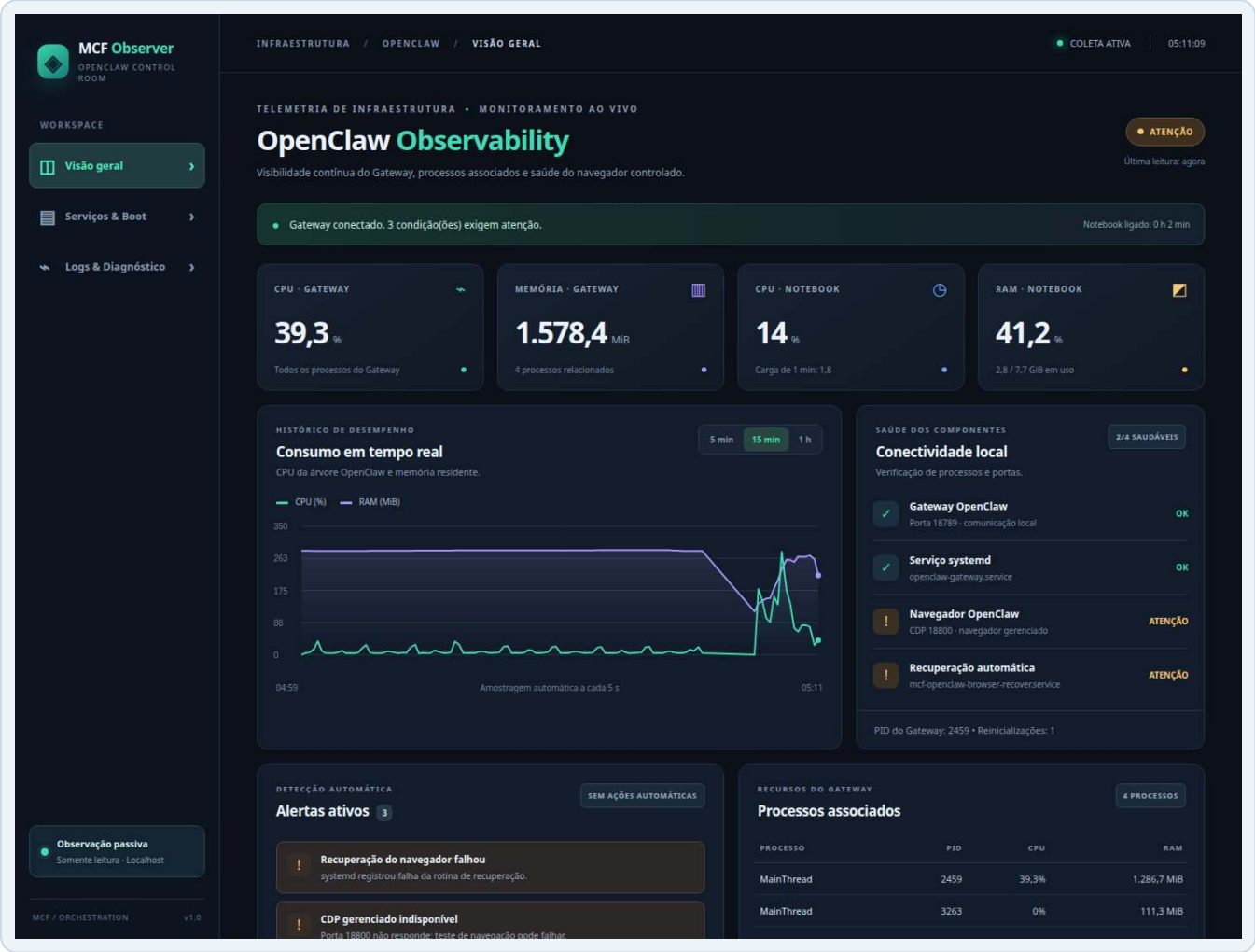


O material original foi gerado com Chrome headless no notebook. Esta versão pública não inclui logs internos brutos.

EVIDÊNCIA VISUAL 02/04 · ORIGEM: CAPTURA REAL

VISÃO GERAL APÓS O REBOOT

Gráfico vivo, Gateway ativo, recursos e condições de atenção.

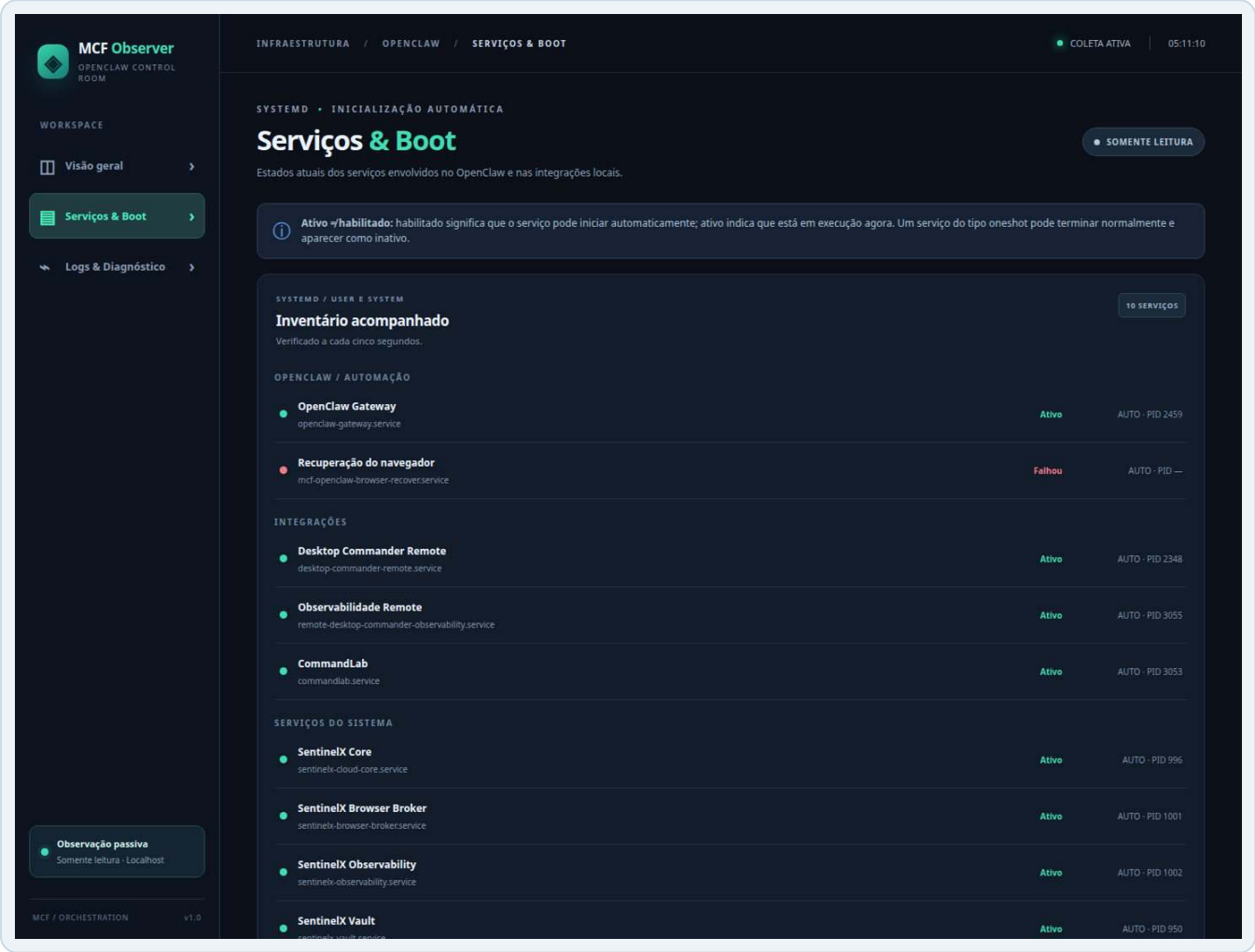


O material original foi gerado com Chrome headless no notebook. Esta versão pública não inclui logs internos brutos.

EVIDÊNCIA VISUAL 03/04 · ORIGEM: CAPTURA REAL

SERVIÇOS E BOOT

Abertura da tela de units systemd com recuperação do navegador marcada como falha.

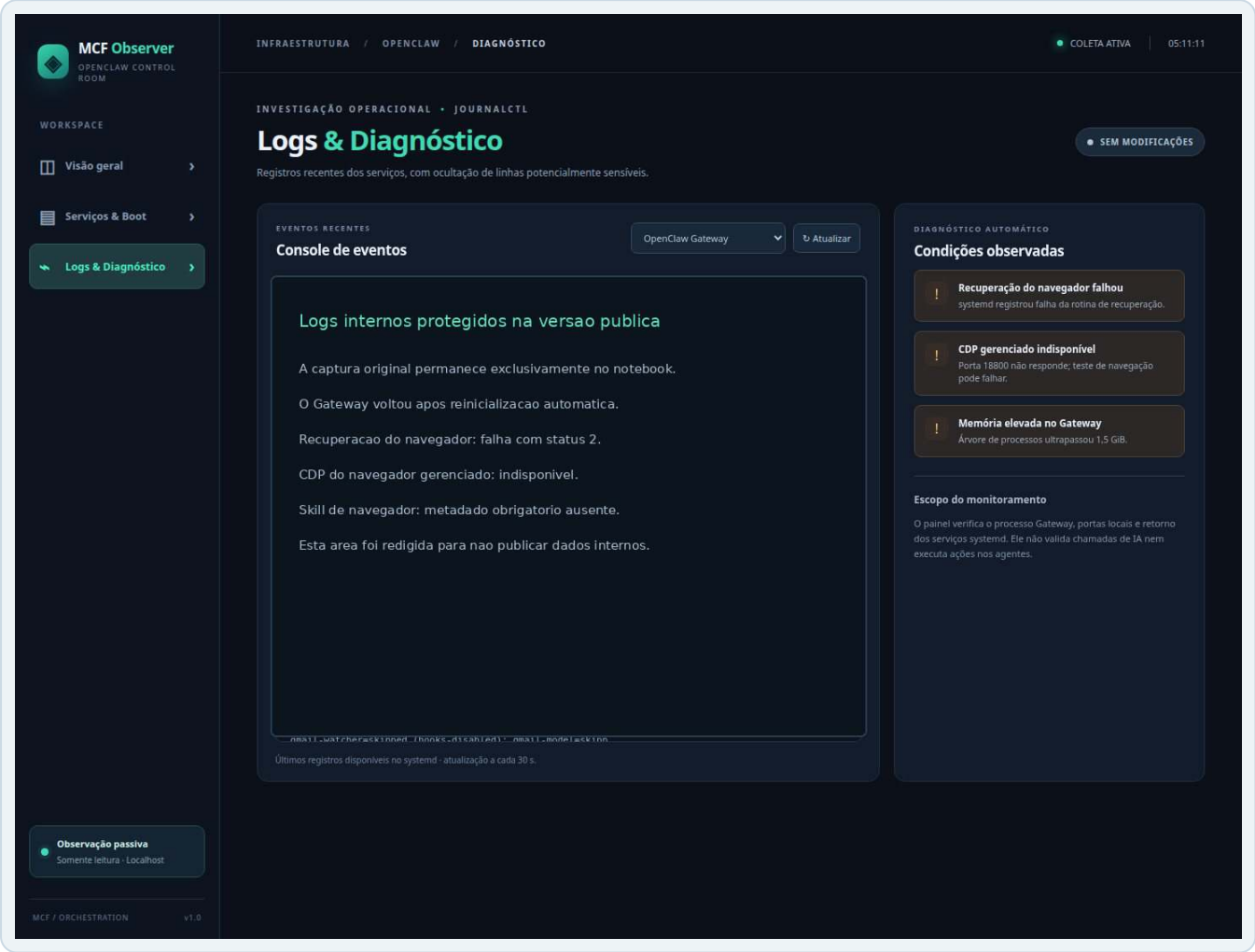


O material original foi gerado com Chrome headless no notebook. Esta versão pública não inclui logs internos brutos.

EVIDÊNCIA VISUAL 04/04 · ORIGEM: CAPTURA REAL

LOGS E DIAGNÓSTICO (REDAÇÃO PÚBLICA)

Visão do diagnóstico; conteúdo bruto do journal oculto nesta cópia pública.



O material original foi gerado com Chrome headless no notebook. Esta versão pública não inclui logs internos brutos.

MCF OBSERVER

Fim do dossiê · estado factual

PAINEL VERIFICADO · GATEWAY OPERACIONAL · BROWSER PENDENTE

A engenharia do observador está descrita, testada e preservada. O Gateway do OpenClaw voltou após retry automático, enquanto o mecanismo de recuperação do navegador permaneceu indisponível.

A etapa de aplicação de correções do subsistema ainda depende de uma missão própria com diagnóstico, plano de reversão e testes. Não foi realizada neste dossiê.

Esta edição pública é um documento estático: não dá controle sobre o computador nem expõe a API local.